

**VIETNAM NATIONAL UNIVERSITY – HO CHI MINH CITY
INTERNATIONAL UNIVERSITY
SCHOOL OF COMPUTER SCIENCE AND ENGINEERING**



**WEB APPLICATIONS DEVELOPMENT
IT093IU
LAB 8**

Member List:

- Huỳnh Ngọc Bảo Long – ITITWE23023

Instructor: MSc. Nguyễn Trung Nghĩa

1. Test 1 – GET all customers

- Method: GET
- URL: /api/customers
- Expected: 200 OK + list of all customers

- Backend logic

- Request reaches:

@GetMapping

```
public ResponseEntity<List<CustomerResponseDTO>> getAllCustomers()
```

- Controller calls service:

```
customerService.getAllCustomers();
```

- Service executes repository method:

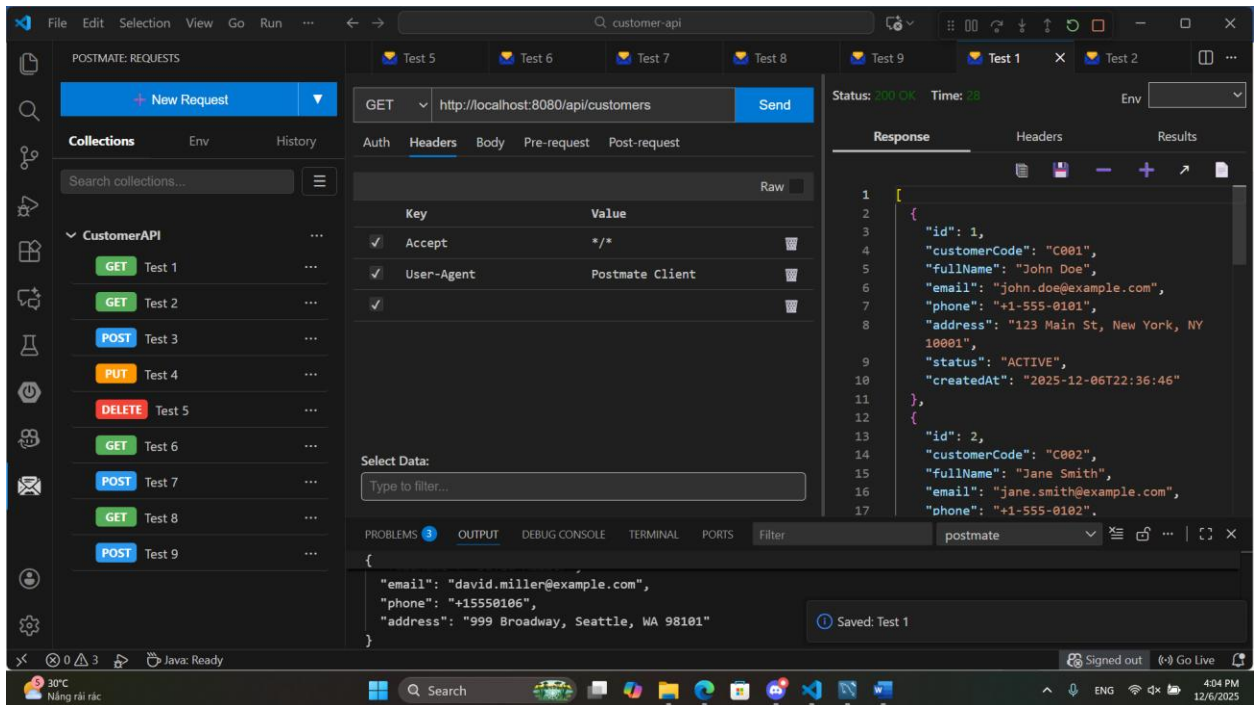
```
return customerRepository.findAll()
```

- Each Customer entity is converted to CustomerResponseDTO using:

```
convertToResponseDTO()
```

- ResponseEntity returns:

```
ResponseEntity.ok(customers);
```

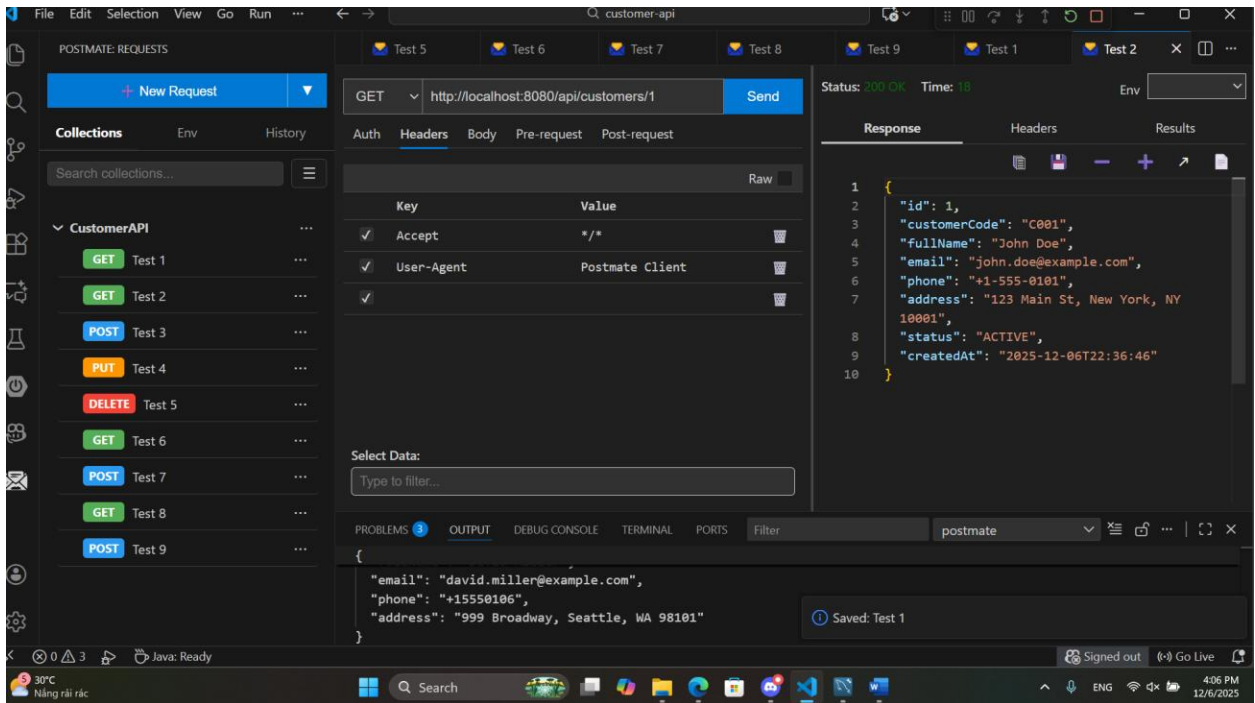


2. Test 2 – GET Customer by ID

- URL: /api/customers/1
- Expected: 200 OK + customer JSON
- Backend logic
 - Controller:

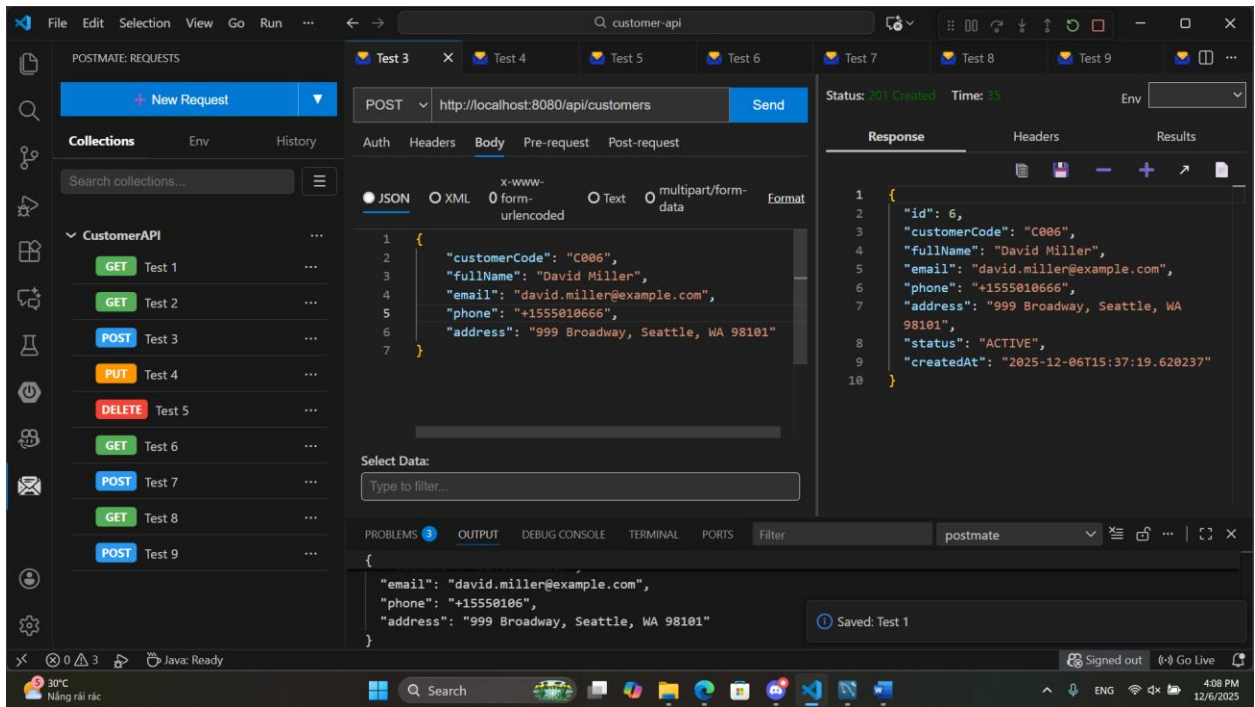

```
@GetMapping("/{id}")
public ResponseEntity<CustomerResponseDTO>
getCustomerById(Long id)
```
 - Service tries to find customer:


```
customerRepository.findById(id)
    .orElseThrow(() -> new ResourceNotFoundException(...));
```
 - If found → converted to DTO → returned.



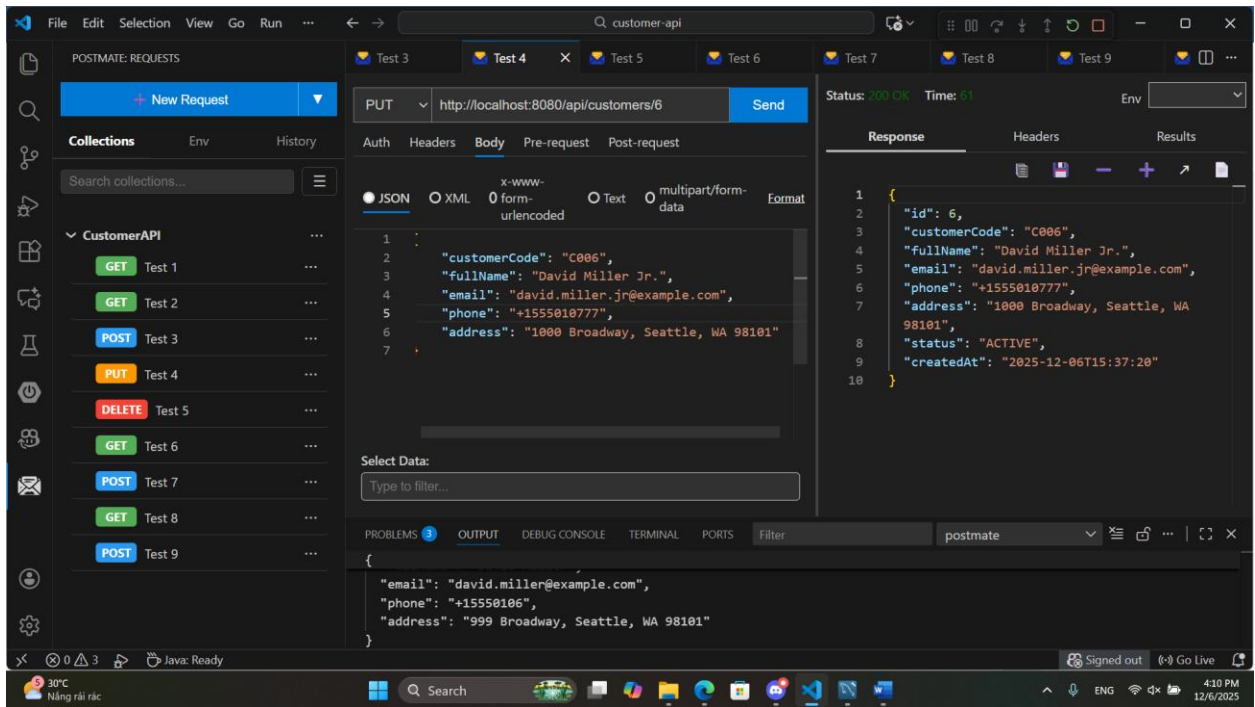
3. Test 3 – POST Create customer

- URL: /api/customers
- Body: valid JSON
- Backend logic
 - Controller validates input using:
@Valid @RequestBody CustomerRequestDTO requestDTO
 - Service checks duplicates:
if existsByCustomerCode(...)
if existsByEmail(...)
 - Converts DTO → Entity
 - Saves new customer:
customerRepository.save(customer)
 - Returned as DTO with 201 CREATED.



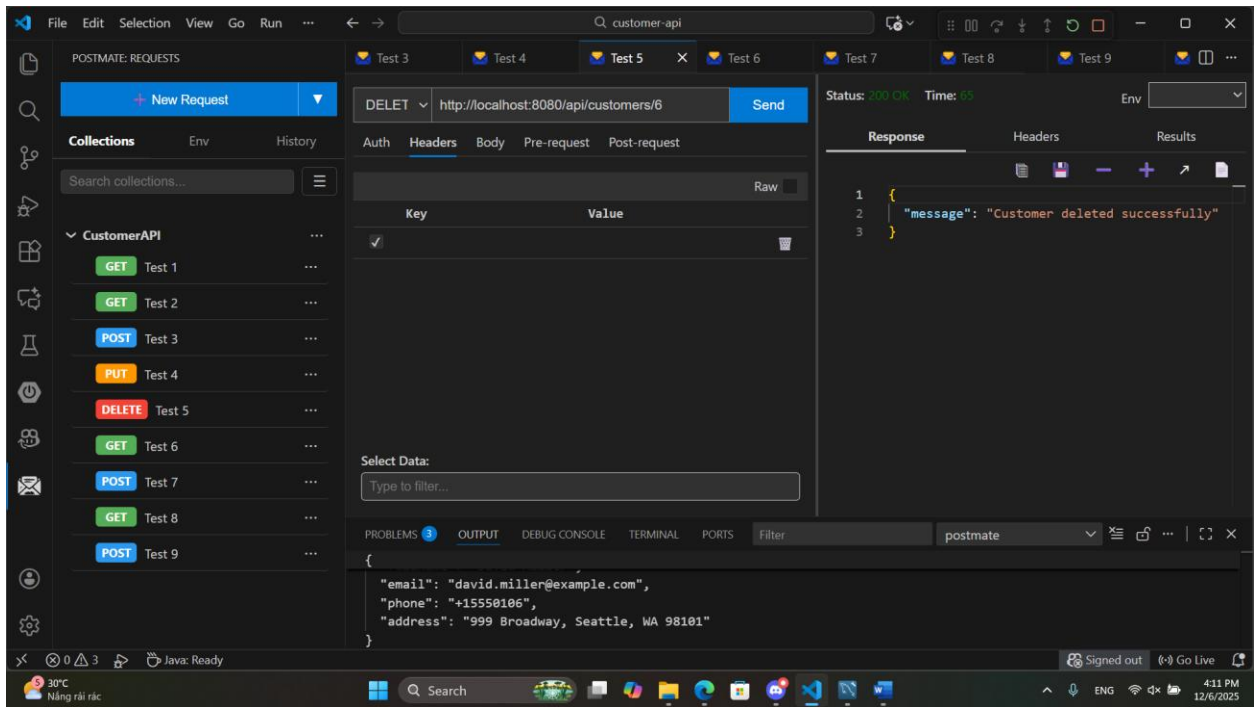
4. Test 4 – PUT update customer

- URL: /api/customers/6
- Body: valid updated fields
- Backend logic
 - Finds existing customer or throws 404.
 - Validates new email:
if email changed AND email already exists → throw DuplicateResourceException
 - Updates fields:
existingCustomer.setFullName(...)
existingCustomer.setEmail(...)
existingCustomer.setPhone(...)
existingCustomer.setAddress(...)
 - Saves updated customer.
 - Returns 200 OK.



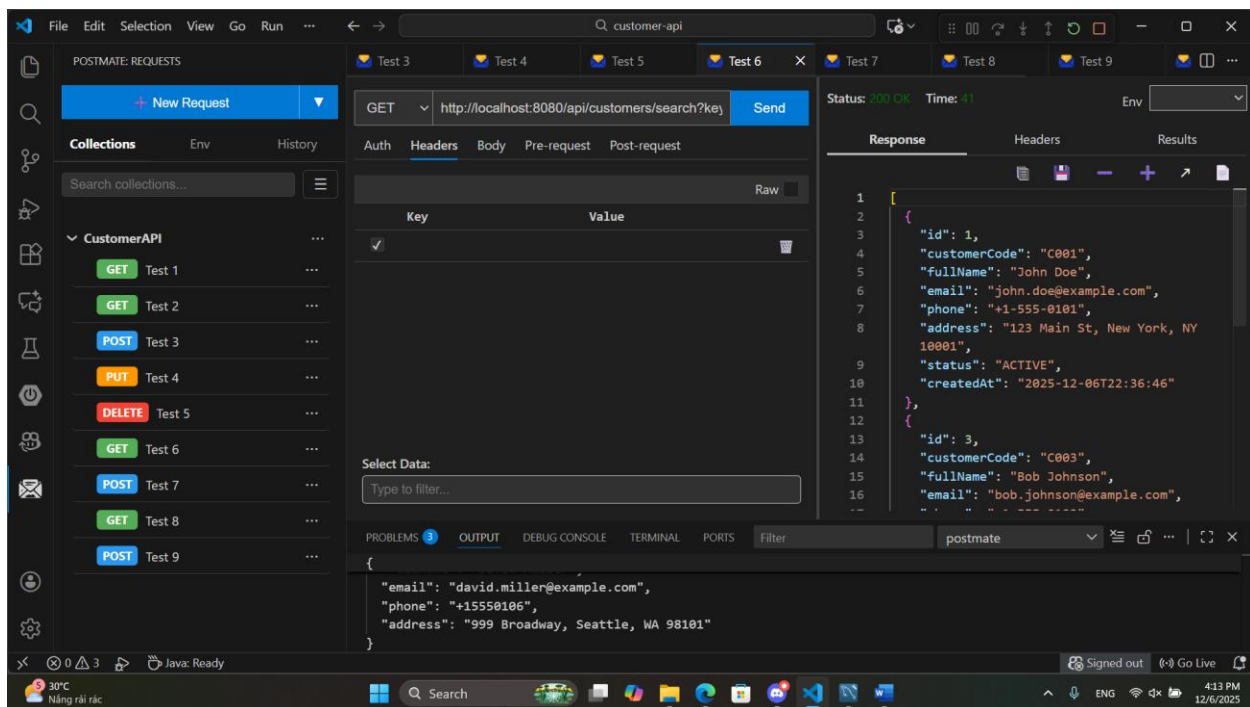
5. Test 5 – DELETE Customer

- URL: /api/customers/6
- Expected: { "message": "Customer deleted successfully" }
- Backend logic
 - Service checks existence:
if (!existsById(id)) throw ResourceNotFoundException
 - Deletes customer:
customerRepository.deleteById(id);
 - Controller returns success message.



6. Test 6 – Search customers

- URL: /api/customers/search?keyword=john
- Expected: list of matching customers
- Backend logic
 - Repository executes JPQL query:
SELECT c FROM Customer c WHERE
LOWER(c.fullName) LIKE %keyword% OR
LOWER(c.email) LIKE %keyword% OR
LOWER(c.customerCode) LIKE %keyword%



7. Test 7 – Validation error

- URL: POST /api/customers
- Body: missing required fields
- Example invalid input


```
{
    "customerCode": "C",
    "email": "invalid-email"
}
```
- Backend logic
 - DTO has validation rules:
 - @NotBlank(message="Full name is required")
 - @Size(...)
 - @Email(...)
 - @Pattern(...)
 - Spring catches validation errors and throws:
 - MethodArgumentNotValidException
 - This is handled by:
 - @ExceptionHandler(MethodArgumentNotValidException.class)
 - Global handler builds:

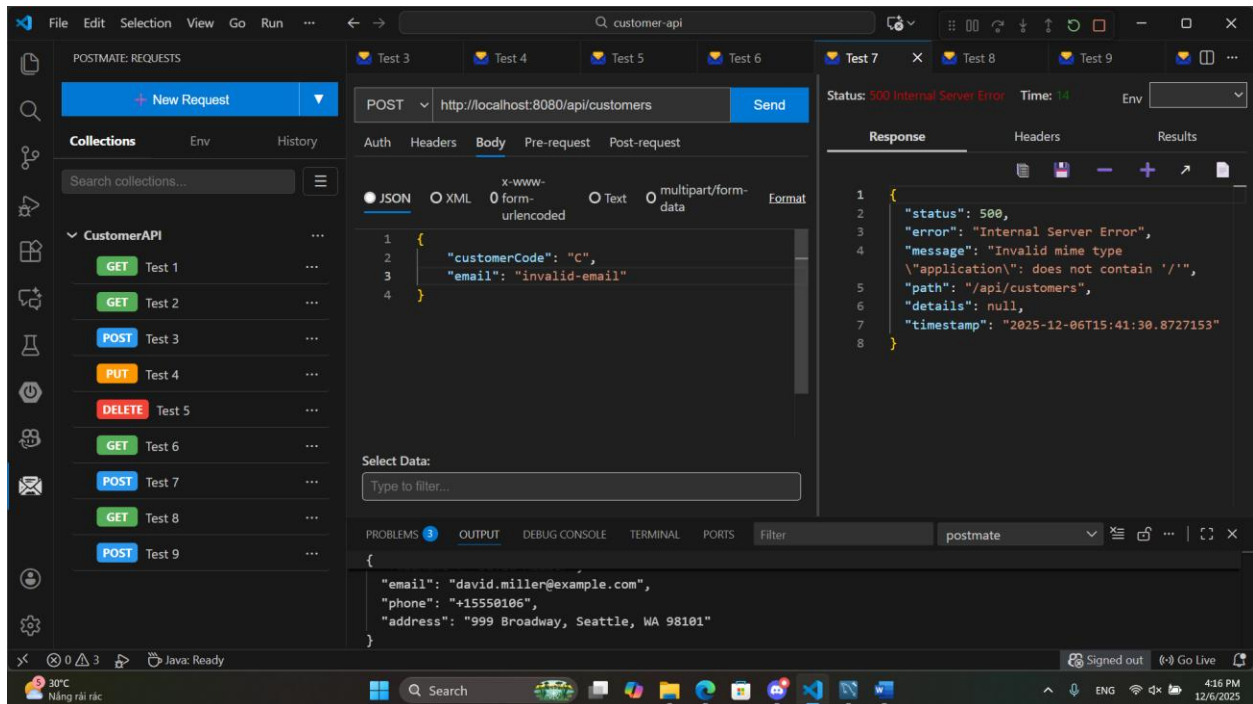

```
{
```



```

    "status": 400,
    "error": "Validation Failed",
    "message": "Invalid input data",
    "details": [...]
  }

```



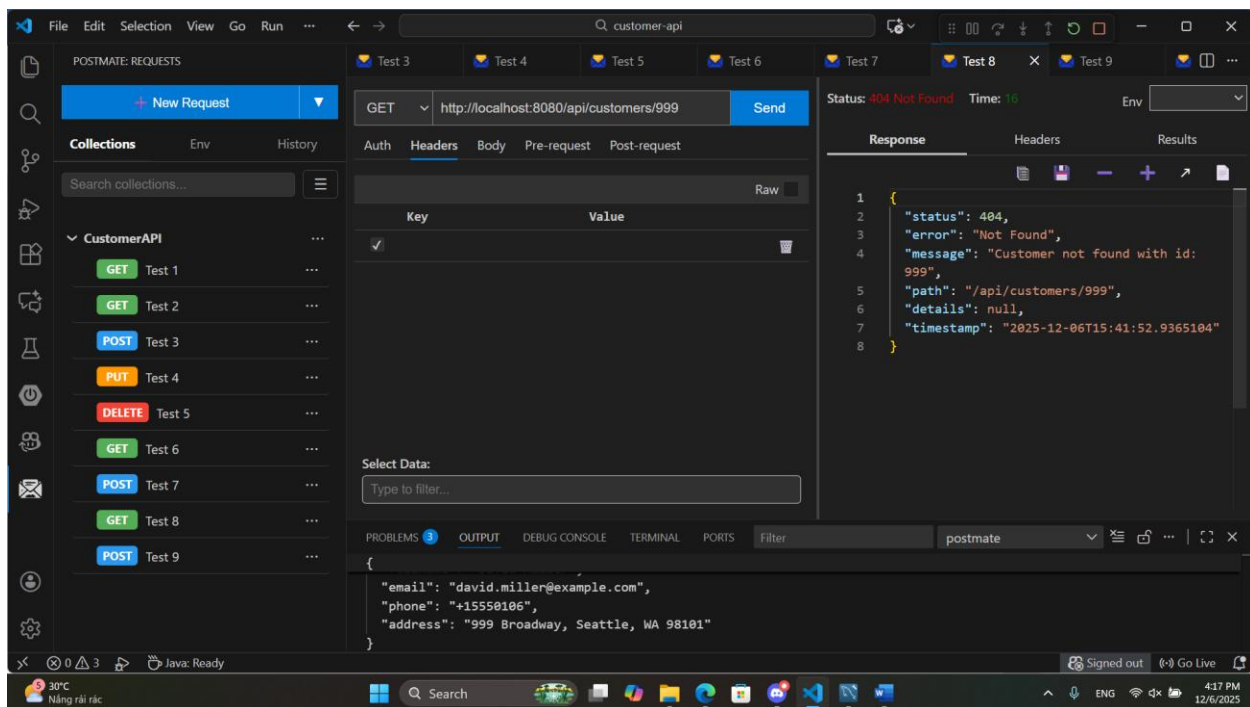
8. Test 8 – Resource not found

- URL: GET /api/customers/999
- Expected: 404 Not Found
- Backend logic
 - Service tries:


```
customerRepository.findById(999)
      .orElseThrow() -> new ResourceNotFoundException(...)
```
 - Exception is caught by handler:


```
@ExceptionHandler(ResourceNotFoundException.class)
```
 - Returns:


```
{
  "status": 404,
  "error": "Not Found",
  "message": "Customer not found with id: 999"
}
```



9. Test 9 – Duplicate resource

- URL: POST /api/customers
- Body: email already exists
- Backend logic:
 - Service checks:


```
if (customerRepository.existsByEmail(requestDTO.getEmail())) {
    throw new DuplicateResourceException("Email already exists: ...");
}
```
 - Global handler catches exception:


```
@ExceptionHandler(DuplicateResourceException.class)
```
 - Returns 409 Conflict:


```
{
  "status": 409,
  "error": "Conflict",
  "message": "Email already exists: john.doe@example.com"
}
```

